

MyBatis批量插入几千条数据，请慎用foreach

来自一个真实项目中的问题，有一个耗时较长的Job存在CPU占用过高的问题，经排查发现，主要时间消耗在往MyBatis中批量插入数据。mapper configuration是用foreach循环做的，差不多是这样。

```
<insert id="batchInsert" parameterType="java.util.List">
    insert into USER (id, name) values
    <foreach collection="list" item="model" index="index" separator=",">
        (#{model.id}, #{model.name})
    </foreach>
</insert>
```

这个方法提升批量插入速度的原理是，将传统的：

```
INSERT INTO `table1` (`field1`, `field2`) VALUES ("data1", "data2");
INSERT INTO `table1` (`field1`, `field2`) VALUES ("data1", "data2");
INSERT INTO `table1` (`field1`, `field2`) VALUES ("data1", "data2");
INSERT INTO `table1` (`field1`, `field2`) VALUES ("data1", "data2");
INSERT INTO `table1` (`field1`, `field2`) VALUES ("data1", "data2");
```

转化为：

```
INSERT INTO `table1` (`field1`, `field2`)
VALUES ("data1", "data2"),
("data1", "data2"),
("data1", "data2"),
("data1", "data2"),
("data1", "data2");
```

在MySQL Docs中也提到过这个trick，如果要优化插入速度时，可以将许多小型操作组合到一个大型操作中。理想情况下，这样可以在单个连接中一次性发送许多新行的数据，并将所有索引更新和一致性检查延迟到最后才进行。

乍看上去这个foreach没有问题，但是经过项目实践发现，当表的列数较多（20+），以及一次性插入的行数较多（5000+）时，整个插入的耗时十分漫长，达到了14分钟，这是不能忍的。在资料中也提到了一句话：

Of course don't combine ALL of them, if the amount is HUGE. Say you have 1000 rows you need to insert, then don't do it one at a time. You shouldn't equally try to have all 1000 rows in a single query. Instead break it into smaller sizes.

它强调，当插入数量很多时，不能一次性全放在一条语句里。可是为什么不能放在同一条语句里呢？这条语句为什么会耗时这么久呢？我查阅了资料发现：

Insert inside Mybatis foreach is not batch, this is a single (could become giant) SQL statement and that brings drawbacks:

some database such as Oracle here does not support.

in relevant cases: there will be a large number of records to insert and the database configured limit (by default around 2000 parameters per statement) will be hit, and eventually possibly DB stack error if the statement itself become too large.

Iteration over the collection must not be done in the mybatis XML. Just execute a simple Insertstatement in a Java Foreach loop. The most important thing is the session Executor type.

```
SqlSession session = sessionFactory.openSession(ExecutorType.BATCH);
for (Model model : list) {
    session.insert("insertStatement", model);
}
session.flushStatements();
```

Unlike default ExecutorType.SIMPLE, the statement will be prepared once and executed for each record to insert.

从资料中可知，默认执行器类型为Simple，会为每个语句创建一个新的预处理语句，也就是创建一个PreparedStatement对象。在我们的项目中，会不停地使用批量插入这个方法，而因为MyBatis对于含有的语句，无法采用缓存，那么在每次调用方法时，都会重新解析sql语句。

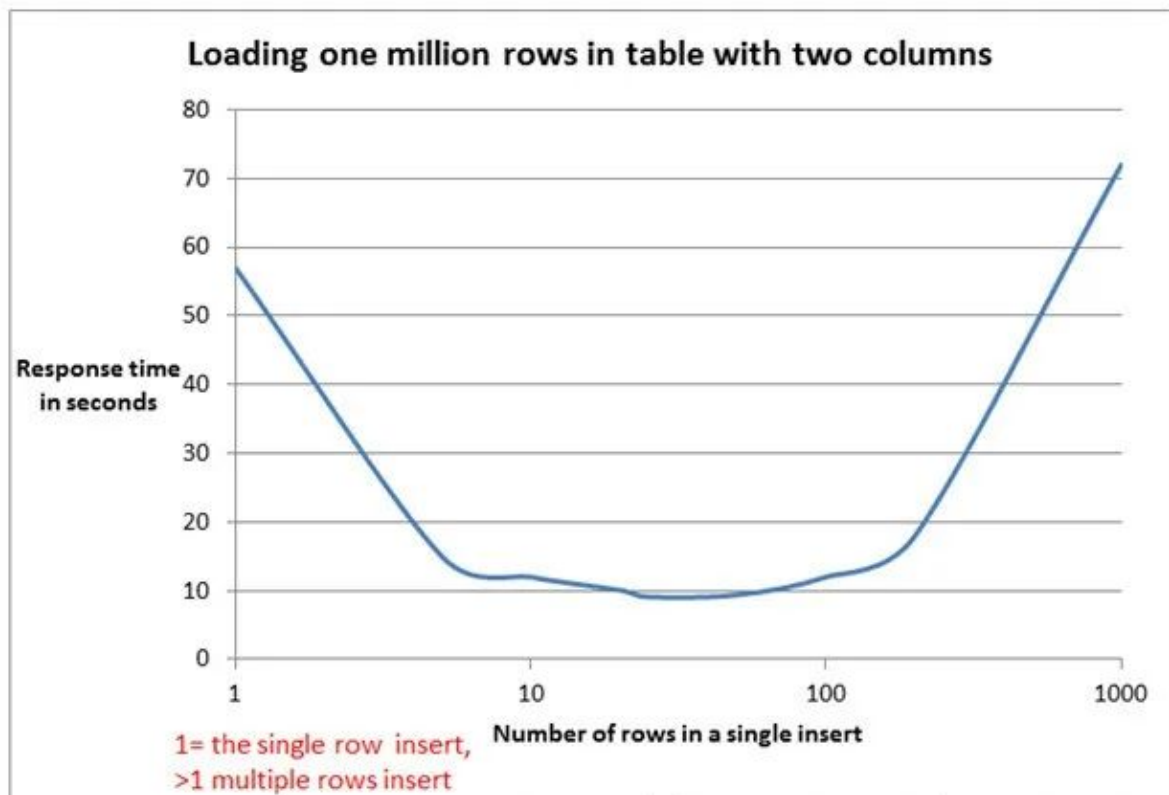
Internally, it still generates the same single insert statement with many placeholders as the JDBC code above.

MyBatis has an ability to cache PreparedStatement, but this statement cannot be cached because it contains element and the statement varies depending on the parameters. As a result, MyBatis has to 1) evaluate the foreach part and 2) parse the statement string to build parameter mapping [1] on every execution of this statement.

And these steps are relatively costly process when the statement string is big and contains many placeholders.

[1] simply put, it is a mapping between placeholders and the parameters.

从上述资料可知，耗时就耗在，由于我foreach后有5000+个values，所以这个PreparedStatement特别长，包含了很多占位符，对于占位符和参数的映射尤其耗时。并且，查阅相关资料可知，values的增长与所需的解析时间，是呈指数型增长的。



所以，如果非要使用 `foreach` 的方式来进行批量插入的话，可以考虑减少一条 `insert` 语句中 `values` 的个数，最好能达到上面曲线的最底部的值，使速度最快。一般按经验来说，一次性插20~50行数量是比较合适的，时间消耗也能接受。

重点来了。上面讲的是，如果非要用 `<foreach>` 的方式来插入，可以提升性能的方式。而实际上，MyBatis文档中写批量插入的时候，是推荐使用另外一种方法。（可以看 <http://www.mybatis.org/mybatis-dynamic-sql/docs/insert.html> 中 Batch Insert Support 标题里的内容）

```
SqlSession session = sqlSessionFactory.openSession(ExecutorType.BATCH);
try {
    SimpleTableMapper mapper = session.getMapper(SimpleTableMapper.class);
    List<SimpleTableRecord> records = getRecordsToInsert(); // not shown

    BatchInsert<SimpleTableRecord> batchInsert = insert(records)
        .into(simpleTable)
        .map(id).toProperty("id")
        .map(firstName).toProperty("firstName")
        .map.lastName).toProperty("lastName")
        .map(birthDate).toProperty("birthDate")
        .map(employed).toProperty("employed")
        .map(occupation).toProperty("occupation")
        .build()
        .render(RenderingStrategy.MYBATIS3);

    batchInsert.insertStatements().stream().forEach(mapper::insert);

    session.commit();
} finally {
    session.close();
}
```

即基本思想是将 MyBatis session 的 executor type 设为 Batch，然后多次执行插入语句。就类似于 JDBC 的下面语句一样。

```
Connection connection =
DriverManager.getConnection("jdbc:mysql://127.0.0.1:3306/mydb?
useUnicode=true&characterEncoding=UTF-
8&useServerPrepStmts=false&rewriteBatchedStatements=true","root","root");
connection.setAutoCommit(false);
PreparedStatement ps = connection.prepareStatement(
    "insert into tb_user (name) values(?)");
for (int i = 0; i < stuNum; i++) {
    ps.setString(1,name);
    ps.addBatch();
}
ps.executeBatch();
connection.commit();
connection.close();
```

经过试验，使用了 ExecutorType.BATCH 的插入方式，性能显著提升，不到 2s 便能全部插入完成。

总结一下，如果MyBatis需要进行批量插入，推荐使用 ExecutorType.BATCH 的插入方式，如果非要使用 `<foreach>` 的插入的话，需要将每次插入的记录控制在 20~50 左右。